

TP Neo4j

Cours + Pratique

De zéro à un projet fil rouge CityFlow, avec Neo4j Browser comme interface

Module	NoSQL — B3 INFO Tronc Commun
Séance	Séance 5 (Jour 3 matin)
Durée estimée	4 à 5 heures (cours + TP)
Outils requis	Docker Desktop, navigateur web, éditeur de texte
Public	Étudiants B3 INFO (INFRES + CYBSEC)
Pré-requis	TP MongoDB + Redis + Cassandra terminés
Formateur	KAKE Abdoulaye
Année	2025 / 2026

Comment utiliser ce document ? Ce TP combine cours et pratique en alternance. « COURS » présente les concepts, « VOTRE TOUR » sont des exercices, « CityFlow » alimente le projet fil rouge. Neo4j est sans doute le plus visuel des 4 outils du module — exploitez les graphes que Neo4j Browser dessine automatiquement.

Pourquoi un modèle graphe ?

Le paradoxe du « relationnel »

Le modèle relationnel s'appelle « relationnel » non pas parce qu'il gère bien les relations entre entités, mais parce qu'il s'appuie sur le concept mathématique de relation (au sens de théorie des ensembles). Paradoxalement, **il est mal adapté aux relations complexes entre données**.

Exemple : sur un réseau social, trouver les « amis d'amis d'amis » d'Alice qu'elle ne connaît pas encore. En SQL, c'est faisable mais avec des auto-jointures qui deviennent prohibitivement lentes à mesure que la profondeur augmente.

```
-- En SQL, "amis d'amis d'amis" devient un cauchemar :
SELECT DISTINCT f3.friend_id
FROM friends f1
JOIN friends f2 ON f1.friend_id = f2.user_id
JOIN friends f3 ON f2.friend_id = f3.user_id
LEFT JOIN friends direct
  ON direct.user_id = f1.user_id AND direct.friend_id = f3.friend_id
WHERE f1.user_id = 'alice'
  AND direct.friend_id IS NULL
  AND f3.friend_id != 'alice';
-- → 3 JOINS, et ça se dégrade exponentiellement avec la profondeur.
```

```
// En Cypher, c'est trivial :
MATCH (alice:User {name: "Alice"})-[:FRIEND_OF*3]-(distant)
WHERE NOT (alice)-[:FRIEND_OF]-(distant)
  AND alice <> distant
RETURN DISTINCT distant.name;
-- → Lisible, performant, peut aller à *4, *5, *6 sans effort.
```

Quand le modèle graphe brille

- **Réseaux sociaux** : amis, followers, recommandations entre utilisateurs
- **Détection de fraude** : trouver les patterns suspects dans les transactions
- **Systèmes de recommandation** : « les utilisateurs qui ont aimé X ont aussi aimé Y »
- **Gestion d'identité (IAM)** : permissions hiérarchiques, groupes, rôles
- **Graphes de connaissances** : ontologies, données liées (Linked Data)
- **Réseaux de transport** : itinéraires, points d'intérêt connectés
- **Analyse d'impact** : « si ce composant tombe, quels services sont affectés ? »
- **GraphRAG** : augmentation des LLM avec des graphes de connaissances

Quand NE PAS utiliser un graphe

- Données tabulaires simples sans relations complexes (utilisez SQL)
- Volumes massifs de données plates (utilisez Cassandra ou un data warehouse)

- Besoins d'écritures à très haut débit (les graphes scalent moins bien que les KV)
- Recherche full-text intensive (utilisez Elasticsearch)

Neo4j : origines et positionnement

Une genèse suédoise

Neo4j a été créé en 2007 par **Emil Eifrem, Johan Svensson et Peter Neubauer** en Suède. Frustrés par les limites des bases relationnelles pour modéliser leur application de gestion de contenu, ils ont créé leur propre moteur. Le « 4j » du nom vient de « for Java » (la version initiale était écrite en Java, et l'est toujours).

Neo4j est devenue la **base graphe la plus utilisée au monde** (selon DB-Engines), avec des clients comme eBay, NASA, Walmart, Airbnb. La société Neo4j Inc. propose une édition Community (gratuite, open-source) et une édition Enterprise (payante, avec clustering avancé). Pour ce cours, nous utilisons la Community Edition via Docker.

Positionnement technique

Modèle	Property Graph : nœuds étiquetés + relations typées orientées + propriétés
Position CAP	CP — Neo4j privilégie la cohérence forte (ACID complète)
Transactions	ACID natif depuis toujours — c'est l'une des forces de Neo4j
Langage	Cypher : déclaratif, lisible, « ASCII art » des graphes — standardisé en GQL (ISO/IEC 39075:2024)
Stockage	Index-free adjacency : chaque nœud sait où sont ses voisins (lookup en $O(1)$)
Scalabilité	Plutôt verticale (1 grosse machine). Clustering en Enterprise avec réplicas en lecture
Cas typiques	Réseaux sociaux, fraude, recommandations, IAM, knowledge graph, GraphRAG
Version actuelle	Neo4j 5 (utilisée dans ce cours)

Index-free adjacency : le secret de la perf

La force technique de Neo4j repose sur l'**index-free adjacency**. Dans une base relationnelle, suivre une relation passe par un lookup d'index (logarithmique). Dans Neo4j, chaque nœud contient des **pointeurs directs** vers ses voisins. Naviguer d'un nœud à un autre est en $O(1)$ — peu importe la taille totale du graphe.

Résultat : faire une traversée à 5 niveaux de profondeur sur un graphe de 1 milliard de nœuds prend le même temps que sur un graphe de 1000 nœuds. C'est ce qui rend Neo4j radicalement plus rapide que SQL pour les requêtes relationnelles complexes.

Partie 1 — Mise en route avec Docker et Neo4j Browser

1.1. Objectif

À la fin de cette partie, vous aurez **Neo4j qui tourne dans Docker** et **Neo4j Browser** accessible depuis votre navigateur. C'est la dernière base à ajouter à votre stack CityFlow, et la plus simple à installer car Neo4j Browser est intégré dans l'image officielle.

1.2. Pourquoi Neo4j ? Pourquoi Neo4j Browser ?

Neo4j est la base de données **orientée graphe** la plus populaire au monde. Là où les autres bases stockent des entités séparées avec des références, Neo4j stocke **explicitement les relations** comme objets de première classe. C'est l'outil parfait pour les réseaux sociaux, les recommandations, la détection de fraude, les itinéraires de transport, ou les graphes de connaissances.

Neo4j Browser est l'interface officielle, **incluse dans l'image Docker**. Pas besoin d'un service séparé — il suffit d'exposer un port supplémentaire et le navigateur fait le reste. C'est aussi l'outil le plus visuel des 4 que vous aurez utilisés : il **dessine vos graphes automatiquement**.

1.3. Étape 1 : continuer avec votre dossier cityflow-nosql

```
cd ./cityflow-nosql
docker compose ps # voir l'état actuel
```

Vous devriez voir vos 6 conteneurs précédents (Mongo + mongo-express, Redis + redis-commander, Cassandra + DataStax Studio). On va ajouter le 7e (et dernier !) service.

1.4. Étape 2 : ajouter Neo4j au docker-compose.yml

Ouvrez votre docker-compose.yml. Après datastax-studio, **avant la section `volumes`**, ajoutez le nouveau service :

```
neo4j:
  image: neo4j:5-community
  container_name: cityflow-neo4j
  restart: unless-stopped
  ports:
    - "7474:7474" # HTTP — interface Neo4j Browser
    - "7687:7687" # Bolt — protocole binaire (drivers)
  volumes:
    - neo4j_data:/data
    - neo4j_logs:/logs
  environment:
    NEO4J_AUTH: neo4j/cityflow2025
    NEO4J_dbms_memory_heap_max__size: 512M
    NEO4J_dbms_memory_pagecache_size: 256M
    NEO4J_PLUGINS: ["apoc"]
```

```
networks:
  - cityflow-net
```

Modifiez la section volumes pour ajouter neo4j_data et neo4j_logs :

```
volumes:
  mongo_data:
  redis_data:
  cassandra_data:
  neo4j_data:
  neo4j_logs:
```

Comprendre la configuration

- ``image: neo4j:5-community`` : version 5 communautaire, gratuite et complète
- ``7474:7474`` : port HTTP pour l'interface Browser
- ``7687:7687`` : port Bolt, protocole binaire utilisé par les drivers applicatifs
- ``NEO4J_AUTH: neo4j/cityflow2025`` : identifiants par défaut (user neo4j, password cityflow2025)
- ``heap_max_size: 512M`` : limite Java heap pour économiser la RAM en local
- ``pagecache_size: 256M`` : cache des pages disque
- ``NEO4J_PLUGINS: ["apoc"]`` : active **APOC**, librairie de fonctions utilitaires (très utile)

Bon à savoir : la **Community Edition** suffit largement pour ce TP et pour la majorité des projets. L'**Enterprise Edition** (payante) ajoute clustering avancé, multi-databases, RBAC fin. Pour un projet B3, la Community fait tout ce qu'il faut.

1.5. Étape 3 : lancer la stack complète

```
# Relancer
docker compose up -d

# Suivre les logs Neo4j (démarrage rapide, ~15s)
docker compose logs -f neo4j
# Attendez "Started." puis CTRL+C

# Vérifier l'état
docker compose ps
```

Vous devez avoir maintenant **7 conteneurs Up**. C'est l'état final de votre stack CityFlow !

Containers [Give feedback](#)

Container CPU usage ⓘ

4.43% / 1400% (14 CPUs available)

Container memory usage ⓘ

1.94GB / 7.47GB

Show charts

Only show running containers

		Name	Container ID	Image	Port(s)	CPU (%)	Memory usage	Actions
☐	☐	cityflow-nosql	-	-	-	4.43%	1.94GB / 53.1GB	☐ ☐ ☐
☐	☐	cityflow-redis	3a1bc5add269	redis:7-alpi	6379:6379 ↗	0.54%	9.73MB / 7.6GB	☐ ☐ ☐
☐	☐	cityflow-cassandra	f0efb18e705b	cassandra:	9042:9042 ↗	1.63%	1.3GB / 7.6GB	☐ ☐ ☐
☐	☐	cityflow-mongo	148916609ed5	mongo:7.0	27017:27017 ↗	0.51%	81.25MB / 7.6GB	☐ ☐ ☐
☐	☐	cityflow-neo4j	8be62084fd60	neo4j:5-co	7474:7474 ↗ Show all ports (2)	1.75%	470.3MB / 7.6GB	☐ ☐ ☐
☐	☐	cityflow-cassandra-web	5720302faa5b	ipushc/cas	8083:8083 ↗	0%	9.92MB / 7.6GB	☐ ☐ ☐
☐	☐	cityflow-redis-commander	657116595dcb	rediscomr	8082:8081 ↗	0%	38.46MB / 7.6GB	☐ ☐ ☐
☐	☐	cityflow-mongo-express	f4be7401ab26	mongo-exp	8081:8081 ↗	0%	43.96MB / 7.6GB	☐ ☐ ☐

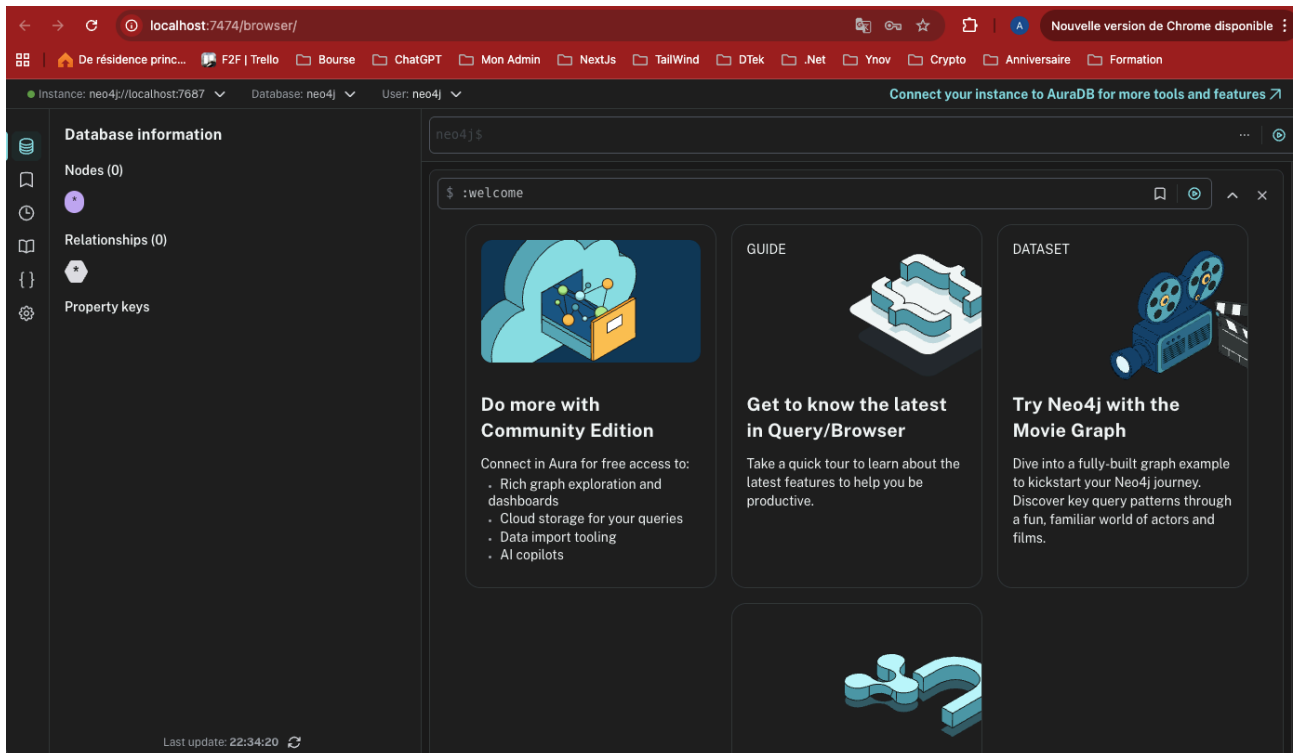
1.6. Étape 4 : ouvrir Neo4j Browser

Dans votre navigateur, ouvrez : **http://localhost:7474**

L'écran d'authentification apparaît. Entrez :

- **Connect URL** : bolt://localhost:7687 (par défaut)
- **Username** : neo4j
- **Password** : cityflow2025
- Cliquez sur **Connect**

Vous arrivez sur l'interface principale de Neo4j Browser. Une base vide vous attend.

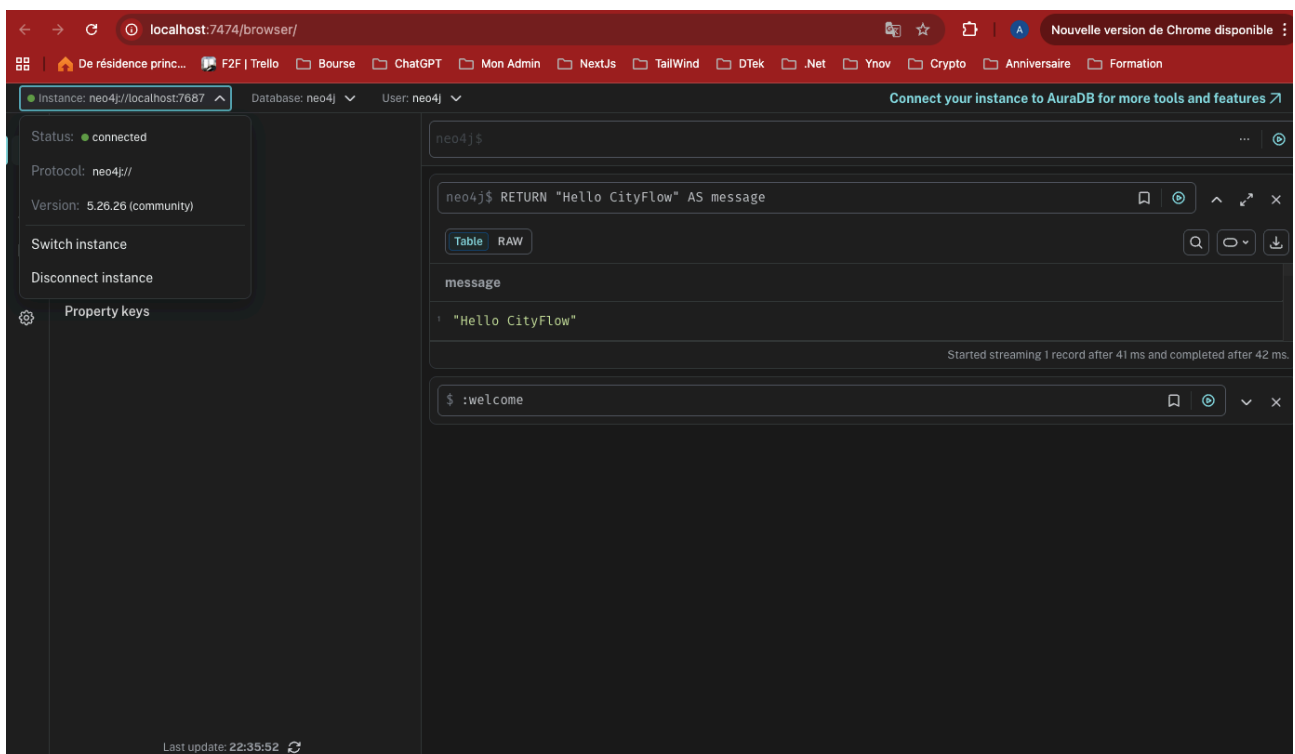


1.7. Étape 5 : tester la connexion

Vous devez voir des informations de connexion. Puis testez une vraie requête :

RETURN "Hello CityFlow" AS message

Vous devez voir un tableau avec une colonne message contenant Hello CityFlow. **Si oui, votre Neo4j est opérationnel.**



VOTRE TOUR — Validation de la mise en route

1. docker compose ps montre **7 conteneurs Up** (les 6 précédents + Neo4j).
2. Toutes les anciennes interfaces marchent toujours :
 - <http://localhost:8081> (mongo-express)
 - <http://localhost:8082> (redis-commander)
 - <http://localhost:9091> (DataStax Studio)
3. <http://localhost:7474> (Neo4j Browser) s'ouvre après authentification.
4. RETURN 'Hello CityFlow' AS message renvoie le bon résultat.

Partie 2 — Découverte de Neo4j Browser

2.1. Anatomie de l'interface

Vue d'ensemble

- **Barre du haut** : champ de saisie de requêtes Cypher (le prompt neo4j\$)
- **Panneau de gauche** : navigation entre labels, types de relations, propriétés
- **Zone centrale** : résultats sous forme de graphe, table, code ou texte
- **Boutons de modes d'affichage** : Graph, Table, Text, Code (en haut à droite du résultat)

Les commandes spéciales (préfixées par `:`)

```
:help      -- aide intégrée  
:clear     -- nettoyer l'historique  
:sysinfo   -- infos système Neo4j
```

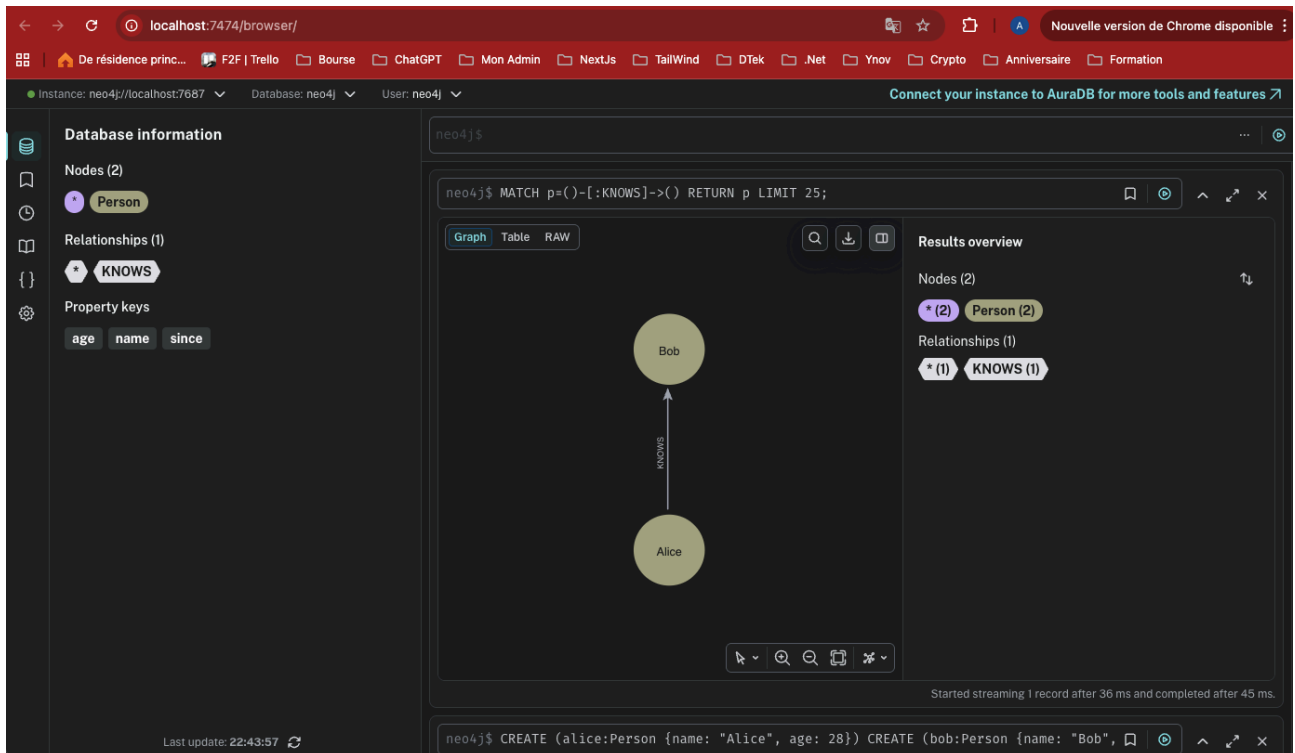
2.2. Le mode graphique : la grande force de Neo4j

Contrairement à MongoDB, Redis ou Cassandra qui affichent des résultats tabulaires ou JSON, **Neo4j Browser dessine automatiquement vos graphes**. Quand votre requête renvoie des nœuds et des relations, vous voyez un véritable diagramme cliquable.

Pour le tester, créons un mini-graphe :

```
CREATE (alice:Person {name: "Alice", age: 28})  
CREATE (bob:Person {name: "Bob", age: 32})  
CREATE (alice)-[:KNOWS {since: 2020}]->(bob)  
RETURN alice, bob
```

Cliquez sur l'onglet `Graph` au-dessus du résultat. Vous voyez deux cercles (Alice et Bob) reliés par une flèche KNOWS. **Cliquez sur un nœud** : ses propriétés s'affichent. **Double-cliquez** : Neo4j développe les relations.



Bienvenue dans la pensée graphe. Vous venez de créer 2 nœuds et 1 relation. C'est la modélisation la plus naturelle qui soit. Aucune table, aucune clé étrangère, aucune jointure : les relations sont des objets à part entière, avec leurs propres propriétés.

2.3. Nettoyer notre base de test

Pour repartir propre avant les vrais exercices :

```
MATCH (n) DETACH DELETE n
-- Supprime tous les nœuds et leurs relations
```

Vérifiez que la base est vide :

```
MATCH (n) RETURN count(n) AS total
-- Doit renvoyer 0
```

Partie 3 — Cours : pourquoi un graphe et le property graph

3.1. Le modèle property graph

Neo4j utilise le modèle **property graph**, défini par 4 éléments :

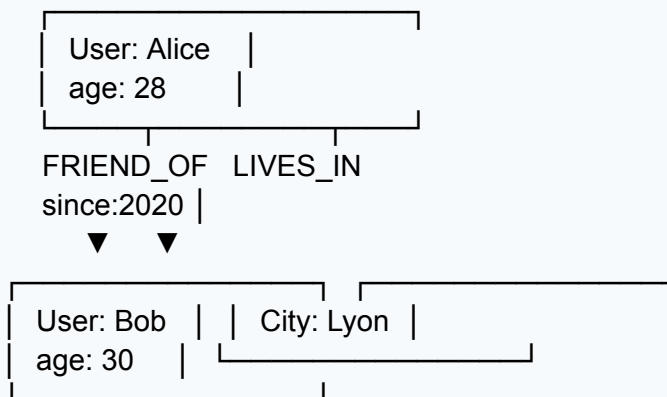
Élément	Description	Exemple
Nœud (Node)	Une entité du domaine	(:User {name: 'Alice', age: 28})
Étiquette (Label)	Une catégorie (équivalent d'une table)	:User, :Product, :Station
Relation (Relationship)	Un lien orienté typé entre deux nœuds	(alice)-[:FRIEND_OF]->(bob)
Propriété (Property)	Une paire clé-valeur sur un nœud ou une relation	{since: '2020-03-15', weight: 0.8}

Caractéristiques importantes

- **Nœuds avec labels multiples** : un nœud peut avoir plusieurs labels (:User:Admin:Premium)
- **Relations toujours orientées** mais traversables dans les deux sens
- **Relations typées** : FRIEND_OF, FOLLOWS, LIKES, etc.
- **Propriétés flexibles** : pas de schéma rigide, on peut ajouter/retirer des propriétés librement
- **Pas de tables, pas de clés étrangères** : les relations sont des objets de première classe

Exemple visuel : un mini réseau social

Exemple de graphe social simple :



Notation Cypher correspondante :

```
(alice:User {name:"Alice", age:28})
(bob:User {name:"Bob", age:30})
(lyon:City {name:"Lyon"})
(alice)-[:FRIEND_OF {since: "2020-03-15"}]->(bob)
(alice)-[:LIVES_IN]->(lyon)
```

Partie 4 — Cours : Cypher, l'art du pattern matching

Cypher est l'idée la plus brillante de Neo4j. Au lieu d'écrire des requêtes procédurales, vous dessinez le motif graphique que vous cherchez avec des caractères ASCII. Cypher est devenu si pertinent qu'il a été standardisé en 2024 sous le nom de GQL (Graph Query Language, ISO/IEC 39075).

4.1. La syntaxe de base : ASCII art

```
// Nœuds : entre parenthèses
(alice)           // nœud anonyme
(alice:User)      // avec un label
(alice:User {name: "Alice"}) // avec propriétés

// Relations : entre crochets, avec flèches pour l'orientation
-[:FRIEND_OF]->  // orientée
-[:FRIEND_OF]-   // non orientée (n'importe quelle direction)
<-[:FRIEND_OF]-  // orientée inverse
-[:FRIEND_OF {since: 2020}]-> // avec propriétés

// Pattern complet
(alice:User)-[:FRIEND_OF]->(bob:User)

// Cypher est CASE-INSENSITIVE pour les mots-clés (MATCH, RETURN...)
// Mais SENSITIVE pour les labels, types de relations et propriétés.
```

4.2. CREATE — créer le graphe

```
// Créer un nœud
CREATE (alice:User {name: "Alice", age: 28, email: "alice@ex.com"})

// Créer plusieurs nœuds et relations en une fois
CREATE (alice:User {name: "Alice"}),
      (bob:User {name: "Bob"}),
      (carol:User {name: "Carol"}),
      (alice)-[:FRIEND_OF {since: "2020"}]->(bob),
      (bob)-[:FRIEND_OF {since: "2021"}]->(carol)

// Créer une relation entre deux nœuds existants
MATCH (a:User {name: "Alice"}), (b:User {name: "Bob"})
CREATE (a)-[:FRIEND_OF]->(b)
```

4.3. MATCH — interroger le graphe

```
// Tous les utilisateurs
MATCH (u:User) RETURN u

// Avec filtre
```

```

MATCH (u:User {name: "Alice"}) RETURN u
// équivalent à :
MATCH (u:User) WHERE u.name = "Alice" RETURN u

// Tous les amis d'Alice
MATCH (alice:User {name: "Alice"})-[:FRIEND_OF]->(friend)
RETURN friend.name

// Amis communs entre Alice et Bob
MATCH (alice:User {name: "Alice"})-[:FRIEND_OF]->(common)<-[:FRIEND_OF]-(bob:User {name: "Bob"})
RETURN common.name

// Compter
MATCH (u:User) RETURN count(u) AS total_users

// Trier et limiter
MATCH (u:User) RETURN u.name, u.age ORDER BY u.age DESC LIMIT 5

```

4.4. MERGE — créer si absent (UPSERT)

MERGE est l'équivalent d'un UPSERT : crée le nœud ou la relation s'il n'existe pas, sinon le retrouve. Très utile pour éviter les doublons lors d'imports.

```

// Créer Alice si elle n'existe pas, sinon la retrouver
MERGE (alice:User {email: "alice@ex.com"})
ON CREATE SET alice.name = "Alice", alice.createdAt = datetime()
ON MATCH SET alice.lastSeen = datetime()
RETURN alice

// Pattern complet en MERGE
MERGE (a:User {email: "alice@ex.com"})
MERGE (b:User {email: "bob@ex.com"})
MERGE (a)-[:FRIEND_OF]->(b)

```

4.5. SET, REMOVE, DELETE

```

// Modifier des propriétés
MATCH (u:User {name: "Alice"}) SET u.age = 29
MATCH (u:User {name: "Alice"}) SET u.age = 29, u.city = "Lyon"

// Ajouter un label
MATCH (u:User {name: "Alice"}) SET u:Premium

// Supprimer une propriété
MATCH (u:User {name: "Alice"}) REMOVE u.phone

// Supprimer un label
MATCH (u:User {name: "Alice"}) REMOVE u:Premium

```

```
// Supprimer un nœud
MATCH (u:User {name: "Alice"}) DELETE u
// Erreur si le nœud a des relations !

// Supprimer un nœud ET ses relations
MATCH (u:User {name: "Alice"}) DETACH DELETE u

// Supprimer une relation
MATCH (alice:User {name: "Alice"})-[r:FRIEND_OF]->(:User {name: "Bob"})
DELETE r
```

4.6. WITH — chaîner les sous-requêtes

WITH est l'équivalent de variables intermédiaires. Il passe le résultat d'une partie de la requête à la suivante, et permet de filtrer ou agréger entre les étapes.

```
// WITH passe le résultat d'une partie à la suivante
MATCH (u:User)
WITH u, size((u)-[:FRIEND_OF]->()) AS friend_count
WHERE friend_count > 5
RETURN u.name, friend_count
ORDER BY friend_count DESC

// Plusieurs étapes
MATCH (alice:User {name: "Alice"})-[r:FRIEND_OF]->(friend)
WITH friend
MATCH (friend)-[:LIKES]->(item)
WITH item, count(*) AS popularity
WHERE popularity >= 2
RETURN item.name, popularity
```

4.7. Recommandations sociales : la magie de Cypher

Le cas d'usage typique : recommander des contacts/produits/contenus basés sur des relations indirectes. C'est là où Cypher brille vraiment.

```
// "Amis d'amis qu'on ne connaît pas" — recommandation sociale classique
MATCH (me:User {name: "Alice"})-[r:FRIEND_OF]-(friend)-[r:FRIEND_OF]-(fof)
WHERE NOT (me)-[:FRIEND_OF]-(fof)
AND me <> fof
RETURN fof.name, count(DISTINCT friend) AS mutual_friends
ORDER BY mutual_friends DESC
LIMIT 10

// "Personnes qui aiment les mêmes choses que moi"
MATCH (me:User {name: "Alice"})-[:LIKES]->(item)-[:LIKES]-(other)
WHERE me <> other
RETURN other.name, count(DISTINCT item) AS common_items
```

```
ORDER BY common_items DESC LIMIT 10
```

```
// "Produits achetés par les acheteurs du produit X"
```

```
MATCH (p:Product {name: "X"})<-[BOUGHT]-(customer)-[:BOUGHT]->(other_p)
```

```
WHERE p <> other_p
```

```
RETURN other_p.name, count(*) AS times_bought
```

```
ORDER BY times_bought DESC LIMIT 5
```

Partie 5 — Exercices de modélisation papier

5.1. Le grand principe : penser en relations

Le réflexe SQL est de penser en tables (entités) puis en clés étrangères (relations). Le réflexe graphe est l'inverse : on commence par identifier les relations métier, puis on attache les nœuds et propriétés autour. C'est un changement de perspective qui prend quelques exercices pour s'installer.

SQL → tables → JOINS	Graphe → relations → traversées
Modéliser les entités d'abord	Modéliser les relations d'abord
Les relations sont des FKs implicites	Les relations sont des objets de première classe
Profondeur 3 = 3 JOINS lents	Profondeur 3 = traversée $O(\text{degré}^3)$ rapide
Schéma rigide à concevoir avant	Schéma évolutif, propriétés ajoutées librement

5.2. Nœud vs propriété : que choisir ?

Question récurrente : faut-il en faire un nœud ou une propriété ? La règle générale :

- **Nœud** si l'entité a sa propre identité, ses propres relations, ou si on veut la chercher comme point de départ d'une requête
- **Propriété** si c'est une donnée descriptive sans existence propre

```
// Exemple : la ville d'un utilisateur
```

```
// Option A — propriété (si on n'a pas besoin de requêter par ville)
(alice:User {name: "Alice", city: "Lyon"})
```

```
// Option B — nœud (si on veut faire "tous les utilisateurs de Lyon")
(alice:User {name: "Alice"})-[:LIVES_IN]->(lyon:City {name: "Lyon"})
```

```
// Si on doit fréquemment faire "tous les utilisateurs de Lyon",
// le nœud séparé est meilleur : on part directement de Lyon.
```

VOTRE TOUR — Exercice 1 : Modéliser LinkedIn

Vous modélisez une plateforme professionnelle type LinkedIn avec :

- Utilisateurs (nom, email, photo)
- Entreprises (nom, secteur, taille)
- Postes occupés (jobs) avec titre, période, entreprise
- Compétences (skills) avec nom
- Relations entre utilisateurs (connexions, mais aussi recommandations professionnelles)

Travail demandé sur papier :

1. Dessinez le graphe : nœuds (avec leurs labels), relations typées (avec leur sens), propriétés sur les uns et les autres.
2. Pour chaque concept (entreprises, postes, compétences) : nœud ou propriété ? Justifiez.
3. Écrivez en Cypher la requête « trouver les collègues actuels d'Alice ».

4. Écrivez la requête « trouver les ex-collègues d'Alice qu'elle n'a pas dans son réseau ».
5. Écrivez la requête « recommander 3 personnes basées sur des compétences communes ».

VOTRE TOUR — Exercice 2 : Catalogue Netflix

Vous modélisez Netflix :

- Utilisateurs avec profil (nom, abonnement)
- Films et séries (titre, année, genres, durée)
- Acteurs et réalisateurs
- Utilisateurs notent les films (rating de 1 à 5)
- Utilisateurs ont une watchlist (films à voir)

Travail demandé :

1. Modélisez le graphe (nœuds, relations, propriétés).
2. Genres : nœuds ou propriétés ? Justifiez selon le besoin de filtrage.
3. Écrivez en Cypher : « films notés 4+ par Alice dans le genre thriller ».
4. Écrivez : « recommander à Alice des films vus et bien notés par les utilisateurs qui aiment les mêmes films qu'elle ». (la fameuse recommandation collaborative !)
5. Écrivez : « les acteurs qui ont joué dans plus de 3 films avec une note moyenne > 4 ».

VOTRE TOUR — Exercice 3 : Réseau de transport (préparation CityFlow)

Vous modélisez un réseau de transport en commun de la métropole de Lyon :

- Stations (nom, code, type : métro, tram, bus, vélo)
- Lignes (numéro, couleur, type)
- Connexions entre stations avec une durée et une distance
- Utilisateurs qui empruntent des trajets (départ, arrivée, date, durée)

Travail demandé :

1. Modélisez le graphe. Réfléchissez : la relation CONNECTED_TO entre deux stations doit-elle être bi-directionnelle ?
2. Comment représenter qu'une ligne **passse par** une station ? Relation ou nœud intermédiaire ?
3. Écrivez la requête « toutes les stations directement connectées à Bellecour ».
4. Écrivez la requête « le plus court chemin (en nombre de stations) entre Bellecour et Croix-Rousse ». (indice : shortestPath, on en parle dans la partie 8)
5. Réfléchissez : comment ajouteriez-vous la durée de trajet à votre modèle pour calculer le **trajet le plus rapide** plutôt que celui avec le moins de stations ?

Partie 6 — Création et insertion guidée (Cypher)

6.1. Objectif

Vous allez créer votre premier graphe complet dans Neo4j Browser et observer comment Neo4j le dessine. La force de Neo4j Browser, c'est que **vous voyez ce que vous créez** en temps réel.

6.2. Étape 1 : nettoyer pour démarrer propre

Dans Neo4j Browser, lancez d'abord :

```
MATCH (n) DETACH DELETE n
```

Vérifiez :

```
MATCH (n) RETURN count(n) AS total
// Doit renvoyer 0
```

6.3. Étape 2 : créer un mini réseau social

On va construire un mini réseau social avec quelques utilisateurs, des amitiés et des intérêts pour des films. Copiez chaque bloc dans Neo4j Browser et exécutez-le.

Création des utilisateurs

```
CREATE (alice:User {name: "Alice", age: 28, city: "Lyon"}),
      (bob:User {name: "Bob", age: 32, city: "Lyon"}),
      (carol:User {name: "Carol", age: 25, city: "Villeurbanne"}),
      (dave:User {name: "Dave", age: 40, city: "Vénissieux"}),
      (eve:User {name: "Eve", age: 22, city: "Lyon"}),
      (frank:User {name: "Frank", age: 35, city: "Caluire"})
```

Création des amitiés

```
MATCH (alice:User {name: "Alice"}), (bob:User {name: "Bob"})
CREATE (alice)-[:FRIEND_OF {since: 2020}]->(bob);

MATCH (alice:User {name: "Alice"}), (carol:User {name: "Carol"})
CREATE (alice)-[:FRIEND_OF {since: 2019}]->(carol);

MATCH (bob:User {name: "Bob"}), (dave:User {name: "Dave"})
CREATE (bob)-[:FRIEND_OF {since: 2021}]->(dave);

MATCH (carol:User {name: "Carol"}), (eve:User {name: "Eve"})
CREATE (carol)-[:FRIEND_OF {since: 2022}]->(eve);

MATCH (dave:User {name: "Dave"}), (frank:User {name: "Frank"})
CREATE (dave)-[:FRIEND_OF {since: 2018}]->(frank);

MATCH (eve:User {name: "Eve"}), (frank:User {name: "Frank"})
```

```
CREATE (eve)-[:FRIEND_OF {since: 2023}]->(frank);
```

Création des films et des likes

```
CREATE (inception:Movie {title: "Inception", year: 2010, genre: "SF"}),  
      (matrix:Movie {title: "The Matrix", year: 1999, genre: "SF"}),  
      (interstellar:Movie {title: "Interstellar", year: 2014, genre: "SF"}),  
      (heat:Movie {title: "Heat", year: 1995, genre: "Thriller"}),  
      (parasite:Movie {title: "Parasite", year: 2019, genre: "Thriller"});
```

```
MATCH (alice:User {name: "Alice"}), (m:Movie {title: "Inception"})  
CREATE (alice)-[:LIKES {rating: 5}]->(m);
```

```
MATCH (alice:User {name: "Alice"}), (m:Movie {title: "The Matrix"})  
CREATE (alice)-[:LIKES {rating: 5}]->(m);
```

```
MATCH (bob:User {name: "Bob"}), (m:Movie {title: "Inception"})  
CREATE (bob)-[:LIKES {rating: 4}]->(m);
```

```
MATCH (bob:User {name: "Bob"}), (m:Movie {title: "Heat"})  
CREATE (bob)-[:LIKES {rating: 5}]->(m);
```

```
MATCH (carol:User {name: "Carol"}), (m:Movie {title: "Interstellar"})  
CREATE (carol)-[:LIKES {rating: 5}]->(m);
```

```
MATCH (carol:User {name: "Carol"}), (m:Movie {title: "Parasite"})  
CREATE (carol)-[:LIKES {rating: 5}]->(m);
```

```
MATCH (dave:User {name: "Dave"}), (m:Movie {title: "The Matrix"})  
CREATE (dave)-[:LIKES {rating: 4}]->(m);
```

6.4. Étape 3 : visualiser le graphe

Affichez la totalité du graphe :

```
MATCH (n) RETURN n LIMIT 100
```

Neo4j Browser dessine le graphe. **Cliquez sur les nœuds** pour les déplacer, voir leurs propriétés. **Cliquez sur les types de relations** en bas pour les masquer/afficher. Vous voyez maintenant le réseau dans son ensemble : c'est ce qui distingue Neo4j des autres bases.

Astuce visualisation : dans le panneau gauche, cliquez sur le label User ou Movie. Vous pouvez personnaliser la couleur des nœuds et leur taille selon une propriété. Très utile pour des démos.

6.5. Étape 4 : observer le schéma émergent

Une particularité de Neo4j : il n'y a pas de schéma rigide à définir avant. Le schéma **émerge** des données que vous créez. Affichez-le :

```
CALL db.schema.visualization()
```

Vous voyez le **méta-graphe** : quels labels existent, quelles relations existent entre quels labels. C'est l'équivalent du diagramme E/R d'une base SQL, mais généré automatiquement.

6.6. Étape 5 : ajouter des contraintes et index

Bien que le schéma soit flexible, on peut imposer des contraintes pour garantir la cohérence et améliorer les performances.

```
// Contrainte d'unicité (et index automatique)
CREATE CONSTRAINT user_name_unique IF NOT EXISTS
FOR (u:User) REQUIRE u.name IS UNIQUE;

CREATE CONSTRAINT movie_title_unique IF NOT EXISTS
FOR (m:Movie) REQUIRE m.title IS UNIQUE;

// Index simple sur une propriété (pour accélérer les recherches)
CREATE INDEX user_city IF NOT EXISTS FOR (u:User) ON (u.city);

// Lister tous les index et contraintes
SHOW INDEXES;
SHOW CONSTRAINTS;
```

Bonne pratique : créez les contraintes d'unicité **avant** d'insérer des données. Sinon vous risquez de découvrir des doublons et la contrainte refusera d'être créée.

VOTRE TOUR — Enrichir votre graphe

Étape 1 — Ajouter 4 utilisateurs supplémentaires

Créez 4 utilisateurs supplémentaires (à votre choix de noms, âges, villes). Utilisez CREATE ou MERGE. Au total vous devez avoir 10 utilisateurs.

Étape 2 — Ajouter des amitiés

Créez **au moins 10 nouvelles amitiés** entre les utilisateurs, en variant les dates (since). Pensez à respecter la cohérence : si Alice est amie avec Bob, Bob est amie avec Alice (relation symétrique).

Étape 3 — Ajouter 3 films + des likes

Créez 3 nouveaux films avec leurs propriétés (titre, année, genre).

Faites en sorte que chaque utilisateur ait noté **au moins 2 films**, avec des ratings variés (de 1 à 5).

Étape 4 — Ajouter un type de nœud `Genre`

Créez les nœuds :Genre et reliez chaque film à son genre via :IN_GENRE. Pourquoi faire ça plutôt que garder le genre comme propriété ? Réfléchissez et notez votre raisonnement dans mes_requetes_cypher.md.

Étape 5 — Validation visuelle

Lancez MATCH (n) RETURN n LIMIT 200 et observez votre graphe complet. Vous devez voir un réseau visiblement dense, avec plusieurs zones interconnectées.

Partie 7 — TP : requêtes Cypher à produire

7.1. Objectif et méthode

Vous allez maintenant **écrire vos propres requêtes Cypher** sur le graphe que vous avez construit. Pour chaque demande, formulez la requête, testez-la dans Neo4j Browser, observez le résultat (en mode Graph et en mode Table), et **notez la requête finale** dans votre fichier `mes_requetes_cypher.md`.

7.2. Exemples résolus pour démarrer

Exemple 1 : tous les utilisateurs d'une ville

```
MATCH (u:User {city: "Lyon"})
RETURN u.name, u.age
ORDER BY u.age DESC
```

Exemple 2 : amis directs d'Alice

```
MATCH (alice:User {name: "Alice"})-[:FRIEND_OF]-(friend)
RETURN friend.name AS amis
// Note : pas de flèche → recherche dans les deux sens
```

Exemple 3 : films notés 4+ par les amis d'Alice

```
MATCH (alice:User {name: "Alice"})-[:FRIEND_OF]-(friend)-[r:LIKES]->(m:Movie)
WHERE r.rating >= 4
RETURN m.title, friend.name, r.rating
ORDER BY r.rating DESC
```

VOTRE TOUR — Requêtes Cypher à produire

Pour chaque demande, **écrivez votre propre requête** Cypher, testez, et notez dans `mes_requetes_cypher.md`.

Niveau 1 — MATCH simples

1. Listez tous les utilisateurs de plus de 25 ans, triés par âge décroissant.
2. Listez les utilisateurs habitant Lyon OU Villeurbanne.
3. Comptez le nombre total d'utilisateurs et le nombre total de films.
4. Listez tous les films de SF, triés par année croissante.

Niveau 2 — Relations simples

5. Listez les amis directs de Carol (avec leur nom et leur ville).
6. Combien d'amis a chaque utilisateur ? Affichez la liste triée par nombre décroissant.
7. Listez les films que Bob a notés, avec leurs notes.
8. Quelle est la note moyenne du film 'Inception' ?

Niveau 3 — Traversées à 2 niveaux

9. Listez les **amis d'amis** d'Alice (à profondeur exactement 2).

10. Listez les films aimés par les amis directs d'Alice, qu'**Alice elle-même n'a pas notés**.
11. Pour chaque utilisateur, listez les genres de films qu'il aime (via :IN_GENRE).

Niveau 4 — Agrégations

12. Top 3 des films les plus populaires (par nombre de likes).
13. Note moyenne par film, triée décroissante.
14. Pour chaque ville, comptez le nombre d'utilisateurs.
15. Utilisateurs ayant noté au moins 3 films.

Niveau 5 — Recommandations

16. **Recommandation par amis communs** : pour Alice, trouvez les utilisateurs avec qui elle a au moins 2 amis communs mais à qui elle n'est pas connectée directement.
17. **Recommandation collaborative simple** : pour Alice, trouvez les films qu'elle n'a pas notés mais qui ont été notés ≥ 4 par au moins 2 de ses amis.
18. **Items populaires entre amis** : trouvez les films aimés par plus de 3 utilisateurs différents et leur note moyenne.

Niveau 6 — Bonus

19. Quel est l'utilisateur le plus connecté du réseau (= celui avec le plus d'amis) ?
20. Existe-t-il une « clique » de 3 utilisateurs tous amis entre eux ? (indice : MATCH (a)-[:FRIEND_OF]-(b)-[:FRIEND_OF]-(c)-[:FRIEND_OF]-(a))

7.3. Conseils en cas de blocage

Si vous bloquez sur une requête :

1. **Dessinez le pattern sur papier** : c'est la grande force de Cypher. Si vous savez le dessiner, vous savez l'écrire.
2. **Commencez petit** : matchez d'abord un seul nœud, ajoutez les relations une par une.
3. **Utilisez RETURN *** pour voir tout ce qui a été matché, puis affinez avec une projection spécifique.
4. **Affichez en mode Graph** : si vous voyez visuellement vos résultats, vous comprenez vite si vous matchez trop ou pas assez.
5. **Cheat sheet en annexe** à la fin du document.

Partie 8 — Cours : traversées, plus court chemin, modélisation avancée

8.1. Profondeur variable : la grande puissance

Cypher permet de rechercher des chemins de **profondeur variable** avec une syntaxe très compacte. C'est ce qui fait sa supériorité sur SQL pour les requêtes relationnelles.

```
// Amis à profondeur 2 exactement
MATCH (alice:User {name: "Alice"})-[:FRIEND_OF*2]-(fof)
RETURN fof.name

// Profondeur de 1 à 3 (amis, amis d'amis, amis d'amis d'amis)
MATCH p = (alice:User {name: "Alice"})-[:FRIEND_OF*1..3]-(person)
RETURN person.name AS Nom, length(p) AS Distance;

// Profondeur 2 à 5
MATCH (alice:User {name: "Alice"})-[:FRIEND_OF*2..5]-(distant)
RETURN distant.name

// Profondeur illimitée (à utiliser avec prudence !)
MATCH (alice:User {name: "Alice"})-[:FRIEND_OF*]-(anyone)
RETURN DISTINCT anyone.name
```

⚠ **Attention** : une profondeur illimitée (*) sur un graphe dense peut faire exploser le temps de calcul. Toujours borner avec *1..N quand vous explorez en profondeur indéterminée.

8.2. shortestPath et allShortestPaths

Pour trouver le **plus court chemin** entre deux nœuds (en nombre de relations), Cypher fournit des fonctions natives.

```
// Le plus court chemin entre Alice et Frank
MATCH path = shortestPath(
  (alice:User {name: "Alice"})-[:FRIEND_OF*]-(frank:User {name: "Frank"})
)
RETURN path, length(path) AS distance

// Tous les plus courts chemins (s'il y en a plusieurs de même longueur)
MATCH paths = allShortestPaths(
  (alice:User {name: "Alice"})-[:FRIEND_OF*]-(frank:User {name: "Frank"})
)
RETURN paths

// Avec contrainte sur la longueur
MATCH path = shortestPath(
  (a:User {name: "Alice"})-[:FRIEND_OF*..6]-(b:User {name: "Frank"})
)
RETURN [n IN nodes(path) | n.name] AS chemin
```

Référence culturelle : la théorie des **6 degrés de séparation** dit que deux humains quelconques sont reliés par au maximum 6 connaissances. C'est exactement ce que `shortestPath *..6` modélise. Sur un graphe réaliste, vous trouvez quasi toujours un chemin en ≤ 6 .

8.3. Algorithmes de graphes : APOC et GDS

APOC (Awesome Procedures On Cypher)

APOC est une librairie de procédures et fonctions très utiles, **livrée d'office avec votre conteneur** (vous l'avez activée via `NEO4J_PLUGINS`). Elle ajoute des centaines de fonctions : conversions, dates, graphes, import/export, etc.

```
// Plus court chemin pondéré (par une propriété de la relation)
MATCH (start:Station {name: "Bellecour"}), (end:Station {name: "Croix-Rousse"})
CALL apoc.algo.dijkstra(start, end, "CONNECTED_TO>", "duration_min")
YIELD path, weight
RETURN path, weight

// Convertir une date
RETURN apoc.date.format(timestamp(), "ms", "yyyy-MM-dd HH:mm")

// Statistiques du graphe
CALL apoc.meta.stats() YIELD labelCount, relTypeCount, nodeCount, relCount
```

GDS (Graph Data Science Library)

Pour les algorithmes avancés (PageRank, communauté de Louvain, centralité, similarité), Neo4j propose la **GDS Library**, plugin séparé. Elle n'est pas activée dans votre TP (plus lourde), mais sachez qu'elle existe pour vos projets sérieux.

8.4. Patterns de modélisation avancée

Pattern 1 : nœud intermédiaire pour les attributs temporels

Quand une relation a beaucoup d'attributs ou doit en avoir d'autres relations vers elle, transformez-la en **nœud-relation**.

```
// Pas terrible si on doit ensuite gérer "qui a annulé la commande"
(:User)-[:ORDERED {date, total, status}]->(:Product)

// Mieux : Order devient un nœud
(:User)-[:PLACED]->(:Order {date, total, status})-[:CONTAINS]->(:Product)
(:Admin)-[:CANCELED]->(:Order)
```

Pattern 2 : modéliser le temps

Pour des analyses temporelles fréquentes, créer un arbre `:Year - :Month - :Day` avec des événements rattachés est très efficace. C'est le **time tree pattern**.

```
// Time tree
(:Year {value:2025})-[:HAS_MONTH]->(:Month {value:9})-[:HAS_DAY]->(:Day {value:15})
```

```
// Connecter un événement
(:Event)-[:HAPPENED_ON]->(:Day {value:15})

// Requête : tous les événements de septembre 2025
MATCH (y:Year {value:2025})-[:HAS_MONTH]->(m:Month
{value:9})-[:HAS_DAY]->(d:Day)<-[:HAPPENED_ON]-(e:Event)
RETURN e, d.value
```

Pattern 3 : héritage avec labels multiples

Un nœud peut avoir plusieurs labels pour représenter une hiérarchie ou des rôles.

```
// Alice est à la fois User, Customer, Premium
CREATE (alice:User:Customer:Premium {name: "Alice"})

// Tous les utilisateurs Premium
MATCH (u:Premium) RETURN u
```

Partie 9 — TP : itinéraires et recommandations

9.1. Objectif

Vous allez appliquer les techniques avancées (profondeur variable, shortestPath) sur des cas d'usage CityFlow concrets : calculer des itinéraires et générer des recommandations.

9.2. TP 1 — Construire un mini-réseau de transport

VOTRE TOUR — Réseau de stations CityFlow

Étape 1 : nettoyer

MATCH (n) DETACH DELETE n

Étape 2 : créer 8 stations de Lyon

Bellecour (B), Cordeliers (C), Hôtel de Ville (H), Croix-Paquet (P), Croix-Rousse (R), Part-Dieu (D), Gare de Vaise (V), Charpennes (CH).

Indice :

```
CREATE (b:Station {code:'B', name:'Bellecour'}),  
      (c:Station {code:'C', name:'Cordeliers'}), ...
```

Étape 3 : créer les connexions avec des durées

Créez les connexions suivantes (bidirectionnelles) avec une duration_min :

- Bellecour ↔ Cordeliers : 2 min
- Cordeliers ↔ Hôtel de Ville : 1 min
- Hôtel de Ville ↔ Croix-Paquet : 2 min
- Croix-Paquet ↔ Croix-Rousse : 3 min
- Hôtel de Ville ↔ Part-Dieu : 4 min
- Bellecour ↔ Part-Dieu : 5 min
- Bellecour ↔ Gare de Vaise : 8 min
- Part-Dieu ↔ Charpennes : 3 min

Indice : pour la bidirectionnalité, créez deux relations dans les deux sens, OU créez une seule relation et matchez sans flèche dans vos requêtes.

Étape 4 : visualiser

MATCH (n) RETURN n

Vous devez voir un graphe en forme d'étoile et de chemin, avec Bellecour et Hôtel de Ville comme nœuds centraux.

9.3. TP 2 — Requêtes d'itinéraires

VOTRE TOUR — Itinéraires

Sur votre réseau de transport, **écrivez les requêtes** suivantes et notez-les dans mes_requetes_cypher.md :

Niveau 1 — Connexions directes

1. Quelles stations sont directement connectées à Bellecour ?
2. Quelle est la durée du trajet direct entre Bellecour et Cordeliers ?

Niveau 2 — Plus court chemin

3. Quel est le plus court chemin (en nombre de stations) entre Bellecour et Croix-Rousse ?

4. Affichez ce chemin sous forme de liste de noms de stations.
5. Trouvez tous les chemins de longueur **exactement 3** entre Bellecour et Charpennes.

Niveau 3 — Plus court chemin pondéré

6. Quel est l'itinéraire **le plus rapide** (en durée) entre Bellecour et Croix-Rousse ?

Niveau 4 — Stations accessibles

7. Quelles stations sont accessibles en **moins de 3 sauts** depuis Bellecour ?
8. Quelles stations sont accessibles en **moins de 10 minutes** depuis Bellecour (somme des durées) ? Indice : c'est plus complexe, utilisez Dijkstra pour chaque station.

Partie 10 — Projet fil rouge CityFlow

10.1. Objectif

Vous avez maintenant toutes les compétences pour intégrer Neo4j dans votre architecture CityFlow. Cette partie est **collective** : faites-la avec votre équipe projet.

10.2. Rappel des user stories Neo4j (N1-N4)

ID	User Story
US-N1	En tant qu'utilisateur, je veux calculer le plus court chemin entre deux stations.
US-N2	En tant qu'utilisateur, je veux trouver les stations accessibles à moins de 15 minutes d'une station donnée.
US-N3	En tant qu'analyste, je veux identifier les stations 'hubs' (les plus connectées) du réseau.
US-N4	En tant qu'utilisateur, je veux trouver un itinéraire sans correspondance entre deux stations (sur une seule ligne).

10.3. Étape 1 — Modélisation collective

VOTRE TOUR — Conception du graphe CityFlow

En équipe, sur papier ou tableau, modélisez le graphe nécessaire aux 4 user stories.

Questions à débattre :

1. **Stations** : un seul label :Station ou plusieurs labels selon le type (:MetroStation, :BusStation, :BikeStation) ?
2. **Lignes** : comment représenter qu'une ligne passe par plusieurs stations dans un ordre précis ? Relation :SERVES avec ordre, ou nœud :LineSegment ?
3. **US-N4** (sans correspondance) : avec votre modèle, comment savoir si deux stations sont sur la même ligne ?
4. **US-N3** (hubs) : quelle propriété d'un nœud représente sa « centralité » ? Le degré (nombre de connexions) ou autre chose ?

Livrable : un schéma dessiné avec :

- Les labels de nœuds (au moins :Station et :Line)
- Les types de relations (:CONNECTED_TO avec quelles propriétés ? :SERVES avec quelles propriétés ?)
- Les contraintes d'unicité à mettre en place

10.4. Étape 2 — Création du graphe CityFlow

VOTRE TOUR — Construction du graphe

Dans Neo4j Browser :

Étape 1 — Nettoyer et créer les contraintes

MATCH (n) DETACH DELETE n;

CREATE CONSTRAINT station_code IF NOT EXISTS FOR (s:Station) REQUIRE s.code IS

```
UNIQUE;  
CREATE CONSTRAINT line_number IF NOT EXISTS FOR (l:Line) REQUIRE l.number IS  
UNIQUE;
```

Étape 2 — Créer un réseau riche

- **15 stations** réparties dans Lyon et sa métropole (utilisez de vrais noms)
- **4 lignes** (Métro A, B, C, D OU des lignes inventées)
- **Au moins 25 connexions** `:CONNECTED\_TO` entre stations (avec durée en minutes)
- **Les relations** `:SERVES` entre chaque ligne et les stations qu'elle dessert (avec ordre)

Étape 3 — Vérifications visuelles

```
MATCH (n) RETURN n LIMIT 200
```

→ Vous devez voir un graphe organisé avec les lignes connectant les stations en chemins.

```
CALL db.schema.visualization()
```

→ Vous voyez le méta-schéma : labels et relations possibles.

10.5. Étape 3 — Implémenter les 4 user stories

VOTRE TOUR — Les requêtes des user stories

Pour chaque user story, **écrivez la requête Cypher** qui la résout. Notez-les dans `queries/neo4j-queries.md` du repo CityFlow.

US-N1 : plus court chemin entre 2 stations

- Avec `shortestPath` pour le minimum de stations.
- Bonus : avec `apoc.algo.dijkstra` pour le minimum en temps.
- Affichez le chemin sous forme de liste lisible de noms.

US-N2 : stations accessibles en < 15 min depuis une station

- Approche simple : `MATCH path = (start)-[:CONNECTED_TO*1..N]-(end)` avec calcul de la somme des `duration_min` sur le chemin.
- Indice : pour calculer la somme, utilisez `reduce(total = 0, r IN relationships(path) | total + r.duration_min)`

US-N3 : stations 'hubs' (les plus connectées)

- Une seule requête avec `count` sur les relations.
- Top 5 par nombre de connexions directes.

US-N4 : itinéraire sans correspondance entre 2 stations

- Trouvez une ligne qui dessert à la fois la station de départ et la station d'arrivée.
- Si oui, c'est faisable sans correspondance.
- Si non, signalez-le.

10.6. Étape 4 — Documenter et commiter

Pour finaliser cette séance et alimenter votre projet CityFlow :

- Dans votre repo Git, créez `docs/modelisation-neo4j.md` avec votre schéma (labels, relations, contraintes) et les choix de modélisation justifiés
- Créez `seed/neo4j/init.cypher` : le script Cypher qui peuple le graphe
- Notez les 4 user stories résolues dans `queries/neo4j-queries.md`
- Faites un commit Git : `feat(neo4j): graphe initial CityFlow et requêtes`

À ce stade, vous avez :

- Une instance Neo4j fonctionnelle dans votre stack CityFlow complète (7 conteneurs !)
- Un graphe de réseau de transport riche et exploitable
- 4 user stories implémentées et documentées
- Un commit Git versionné

C'est la moitié de votre note de projet CityFlow pour la partie Neo4j !

ET surtout : vous avez maintenant terminé l'apprentissage des 4 paradigmes NoSQL majeurs. Votre architecture CityFlow polyglotte est complète : MongoDB pour les profils, Redis pour le cache et les sessions, Cassandra pour les logs massifs, Neo4j pour le réseau de transport.

Annexe A — Cheat sheet Neo4j Browser

Action	Comment ?
Se connecter	http://localhost:7474 (neo4j / cityflow2025)
Exécuter une requête	Tapez dans la barre + Ctrl+Entrée (ou bouton ▶)
Effacer l'historique	:clear
Voir le schéma	CALL db.schema.visualization() ou :schema
Aide intégrée	:help
Guides interactifs	:play start
État du serveur	:server status
Voir les index	SHOW INDEXES
Voir les contraintes	SHOW CONSTRAINTS
Personnaliser un label	Cliquer sur le label en bas → choisir couleur/taille

Annexe B — Mémo des commandes Cypher

B.1. Création

```
CREATE (n:Label {prop:value})
CREATE (a)-[:REL_TYPE {prop:value}]->(b)
MERGE (n:Label {key:value}) ON CREATE SET ... ON MATCH SET ...
```

B.2. Lecture

```
MATCH (n) RETURN n LIMIT 100
MATCH (n:Label {prop:value}) RETURN n
MATCH (n:Label) WHERE n.prop > 100 RETURN n
MATCH (a)-[:REL]->(b) RETURN a, b
MATCH (a)-[:REL*1..3]-(b) RETURN a, b
MATCH path = shortestPath((a)-[*]-(b)) RETURN path
RETURN n ORDER BY n.prop DESC LIMIT 5
```

B.3. Modification

```
MATCH (n) SET n.prop = value
MATCH (n) SET n:NewLabel
MATCH (n) REMOVE n.prop
MATCH (n) REMOVE n:Label
MATCH (n) DELETE n          -- échoue si relations
MATCH (n) DETACH DELETE n   -- supprime avec relations
```

B.4. Agrégation

```
RETURN count(*), count(n.prop)
RETURN avg(n.prop), min(n.prop), max(n.prop), sum(n.prop)
RETURN n.label, count(*) AS total
RETURN collect(n.name) AS noms
```

B.5. Contraintes et index

```
CREATE CONSTRAINT name IF NOT EXISTS FOR (n:Label) REQUIRE n.prop IS UNIQUE
CREATE INDEX name IF NOT EXISTS FOR (n:Label) ON (n.prop)
SHOW CONSTRAINTS
SHOW INDEXES
DROP CONSTRAINT name
DROP INDEX name
```

B.6. Patterns utiles

```
// Pattern non orienté (cherche dans les deux sens)
MATCH (a)-[:REL]-(b)

// Pattern OPTIONAL (renvoie quand même si pas de match)
MATCH (a:User)
OPTIONAL MATCH (a)-[:FRIEND_OF]->(b)
RETURN a, b

// WITH pour chaîner
MATCH (a) WITH a, count(a) AS n WHERE n > 1 RETURN a, n

// UNION pour combiner deux requêtes
MATCH (n:Label1) RETURN n.name UNION MATCH (n:Label2) RETURN n.name
```

Annexe C — Pour aller plus loin

C.1. Connecter Neo4j à du code

```
// Node.js avec le driver officiel
const neo4j = require("neo4j-driver");
const driver = neo4j.driver(
  "bolt://localhost:7687",
  neo4j.auth.basic("neo4j", "cityflow2025")
);
const session = driver.session();
const result = await session.run(
  "MATCH (u:User {name: $name}) RETURN u",
  { name: "Alice" }
);

# Python avec le driver officiel
```

```

from neo4j import GraphDatabase
driver = GraphDatabase.driver("bolt://localhost:7687", auth=("neo4j", "cityflow2025"))
with driver.session() as session:
    result = session.run("MATCH (u:User {name: $name}) RETURN u", name="Alice")

// .NET avec Neo4j.Driver
using Neo4j.Driver;
var driver = GraphDatabase.Driver("bolt://localhost:7687",
    AuthTokens.Basic("neo4j", "cityflow2025"));
var session = driver.AsyncSession();
var result = await session.RunAsync("MATCH (u:User {name: $name}) RETURN u",
    new { name = "Alice" });

```

C.2. GQL : Cypher devient un standard

En avril 2024, **GQL (Graph Query Language)** a été standardisé par l'ISO (ISO/IEC 39075:2024). C'est le premier nouveau standard de requête depuis SQL (1986). GQL est très largement basé sur Cypher, ce qui signifie que **les compétences Cypher que vous acquérez aujourd'hui sont transférables** sur toutes les bases graphe qui supporteront GQL.

C.3. Neo4j Aura : cloud managé

Pour la production, Neo4j propose **Aura**, une offre cloud managée (AWS, GCP, Azure). Démarrage à ~65 €/mois pour une instance de production. Plan gratuit AuraDB Free pour le dev (limité à 200 000 nœuds, 400 000 relations).

C.4. Ressources recommandées

- **Neo4j GraphAcademy** : graphacademy.neo4j.com — cours gratuits avec certifications
- **Documentation officielle** : neo4j.com/docs
- **Neo4j Cypher Manual** : neo4j.com/docs/cypher-manual
- **Graph Databases** (O'Reilly, gratuit en PDF) : neo4j.com/graph-databases-book
- **APOC documentation** : neo4j.com/docs/apoc

À retenir absolument de cette séance

- **Neo4j = property graph** : nœuds étiquetés + relations typées orientées + propriétés partout.
- **Les relations sont des objets de première classe** avec leurs propres propriétés.
- **Cypher se lit en ASCII art** : (a)-[:REL]->(b). Si vous savez le dessiner, vous savez l'écrire.
- **Index-free adjacency** : $O(1)$ pour naviguer d'un nœud à un voisin, peu importe la taille du graphe.
- **Profondeur variable $\sim 1..N$** : la grande puissance pour les requêtes relationnelles complexes.
- **shortestPath** pour le plus court chemin, **apoc.algo.dijkstra** pour le plus court pondéré.
- **ACID complet** : Neo4j a des transactions, ce que ni MongoDB simple ni Cassandra n'ont par défaut.
- **Pensez en relations d'abord** : c'est le saut mental par rapport au modèle relationnel.
- **Modéliser : nœud ou propriété ?** Nœud si l'entité a sa propre identité et ses propres relations.